

Katholieke
Universiteit
Leuven

**Department of
Computer Science**

COALESCED MEMORY ACCESS AND GPU MEMORY TYPES

Lab 4

Matt Delalieux (r0890338)
Radi Miah (r1012080)

Academic year 2025–2026

Contents

1 Part 1 – Coalesced Memory Access **2**
- 1.1 Implementation . . . 2
  - 1.1.1 Matrix Layout . . . 2
  - 1.1.2 Copy Kernel . . . 2
  - 1.1.3 Transpose Kernel . . . 2
- 1.2 Results . . . 2
- 1.3 Analysis . . . 3
  - 1.3.1 Effect of Matrix Size . . . 3
  - 1.3.2 CPU vs. GPU Comparison . . . 4
- 1.4 Conclusions . . . 4

2 Part 2 – Global, Shared, and Constant Memory **4**
- 2.1 Implementation . . . 4
  - 2.1.1 Matrix Multiplication – Overview . . . 4
  - 2.1.2 Kernel 1 – Global Memory Only . . . 4
  - 2.1.3 Kernel 2 – Global + Shared Memory . . . 5
  - 2.1.4 Kernel 3 – Global + Constant Memory . . . 5
- 2.2 Results . . . 6
  - 2.2.1 Memory Migration Times . . . 6
  - 2.2.2 Kernel Execution Times . . . 7
- 2.3 Analysis . . . 9
  - 2.3.1 How Much Does Memory Migration Cost? . . . 9
  - 2.3.2 How Do the Three Memory Types Compare? . . . 9
  - 2.3.3 How Do the Times Scale with Input Size? . . . 9
- 2.4 Conclusions . . . 9

1 Part 1 – Coalesced Memory Access

1.1 Implementation

1.1.1 Matrix Layout

The $N \times N$ matrix is stored as a flat 1D array in row-major order, with element A[row][col] at index row$\times N$+col. This means consecutive threads in a warp naturally access consecutive memory addresses, allowing their requests to be combined into a single memory transaction.

1.1.2 Copy Kernel

Each thread copies one element from input to output using its linear index. Since consecutive threads access consecutive addresses, all memory operations are combined into single transactions, giving fully coalesced access.

```
__global__ void copyMatrix(const int* __restrict__ matrix, int*       __restrict__ result,
    int total_elements)
{
    int abs_tid   = blockIdx.x * blockDim.x + threadIdx.x;
    int grid_size = gridDim.x  * blockDim.x;
    for (int idx = abs_tid; idx < total_elements; idx += grid_size)
        result[idx] = matrix[idx];
}
```

Listing 1: Matrix copy kernel (B = A)

1.1.3 Transpose Kernel

Transposing requires remapping A[row][col] to B[col][row]. This means that either reads or writes must jump around in memory. Here the reads are sequential, but writes land at addresses N elements apart – so they cannot be combined into single transactions.

```
__global__ void transposeKernel(const int* __restrict__ matrix, int*       __restrict__
    result, int N)
{
    int total_elements = N * N;
    int abs_tid        = blockIdx.x * blockDim.x + threadIdx.x;
    int grid_size      = gridDim.x  * blockDim.x;
    for (int idx = abs_tid; idx < total_elements; idx += grid_size) {
        int row = idx / N;
        int col = idx % N;
        result[col * N + row] = matrix[row * N + col];
    }
}
```

Listing 2: Matrix transpose kernel ($B = A^T$)

1.2 Results

All results were verified against CPU-computed reference outputs. Each size was benchmarked over 20 timed runs preceded by one untimed warm-up run; median execution time is reported throughout.

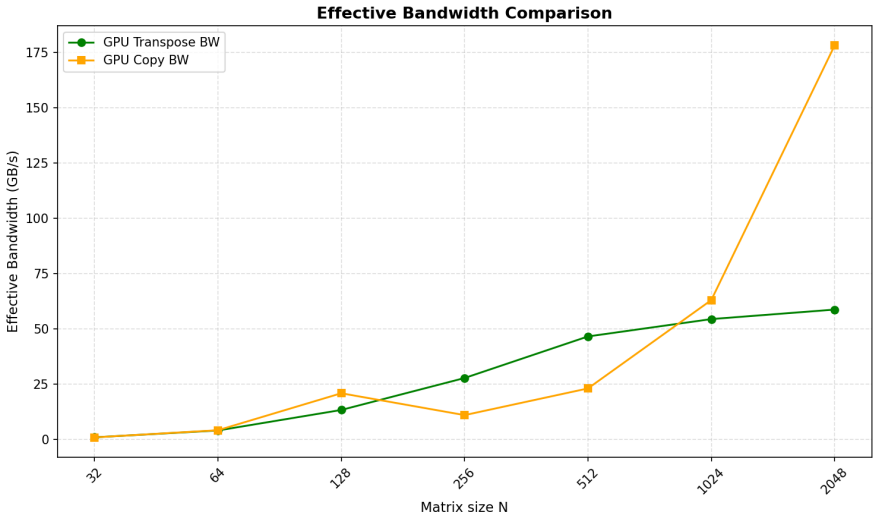


Figure 1: Effective bandwidth (GB/s) vs. matrix size N for the GPU copy and transpose kernels.

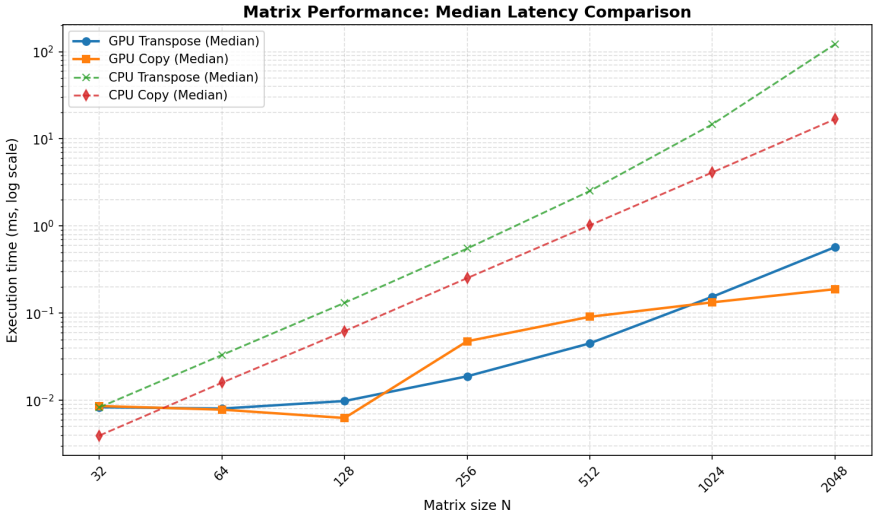


Figure 2: Median execution time (ms, log scale) vs. matrix size N for the GPU and CPU kernels.

Table 1: Effective bandwidth at selected matrix sizes (median over 20 runs, RTX 2070).

N	Copy BW (GB/s)	Transpose BW (GB/s)
512	∼23	∼47
1024	∼63	∼54
2048	∼178	∼59

1.3 Analysis

1.3.1 Effect of Matrix Size

At small sizes ($N \leq 128$) both kernels show very low bandwidth. The matrices are tiny and there is not enough work to keep the GPU busy, so measured time is dominated by the fixed overhead of launching the kernel rather than the actual memory transfers.

As N grows the GPU becomes fully utilised and the two kernels diverge clearly, as seen in Figure 1.

Copy kernel. Consecutive threads access consecutive memory addresses, so their requests are combined into single transactions. Bandwidth scales steeply with N, reaching ∼178 GB/s at $N = 2048$, at intermediate sizes the matrix no longer fits in L2 cache, causing a temporary bandwidth drop before full GPU utilisation is reached at larger N..

Transpose kernel. Although reads are sequential, writes land at addresses N elements apart. Each group of 32 threads therefore issues up to 32 separate write transactions instead of one, wasting most of the available memory bandwidth. As a result, effective bandwidth plateaus at ∼55–60 GB/s from $N = 512$ onwards regardless of how large the matrix gets.

1.3.2 CPU vs. GPU Comparison

The CPU copy is reasonably fast at small N but falls well behind the GPU copy at large N, where the GPU's parallelism gives it a clear advantage.

The CPU transpose is by far the slowest of all four variants. Its loop reads the input matrix one column at a time, jumping N elements with every step. At large N almost every read misses the CPU cache, causing execution time to grow much faster than N^2 – reaching ∼130 ms at $N = 2048$ versus ∼0.5 ms for the GPU transpose.

1.4 Conclusions

The results clearly show that how memory is accessed matters far more than how much data is moved. The copy and transpose kernels transfer exactly the same amount of data, but the copy achieves ∼178 GB/s at $N = 2048$ while the transpose reaches only ∼59 GB/s – roughly 3× lower – purely because its writes are spread across non-adjacent memory locations.

The CPU transpose illustrates the same problem: without the GPU's massive parallelism, strided memory access becomes extremely costly, making it orders of magnitude slower than either GPU kernel at large N.

The standard fix for the transpose is to use shared memory as a staging area: load a tile of the input sequentially, rearrange it on-chip, then write it out sequentially. This eliminates the scattered write pattern and should recover bandwidth close to the copy kernel.

2 Part 2 – Global, Shared, and Constant Memory

2.1 Implementation

2.1.1 Matrix Multiplication – Overview

Each of the N^2 threads computes one output element of C by accumulating N multiply-add operations. Without caching, each row of A and each column of B is re-read from global memory on every iteration.

2.1.2 Kernel 1 – Global Memory Only

Thread `(row, col)` reads `A[row*N+k]` and `B[k*N+col]` straight from global memory for each k. The reads of B are non-coalesced: consecutive threads each need a different row of B, spreading their requests N elements apart – the same scattered-access problem as the transpose kernel in Part 1.

```
__global__ void multiplyMatrix(int* matrixA, int* matrixB, int* result, int n)
{
    int abs_tid = blockIdx.x * blockDim.x + threadIdx.x;
    int row = abs_tid / n;
    int col = abs_tid % n;
    int sum = 0;
    for (int i = 0; i < n; i++)
        sum += matrixA[row * n + i] * matrixB[i * n + col];
    result[abs_tid] = sum;
}
```

Listing 3: Matrix multiplication kernel – global memory only

2.1.3 Kernel 2 – Global + Shared Memory

Each block cooperatively loads a 32×32 tile of A and B into shared memory (a fast on-chip scratchpad), synchronises, then accumulates the partial dot product from those cached values. Repeating this across all tiles reduces global memory reads by $32\times$ compared to Kernel 1, and the cooperative load pattern also fixes the non-coalesced B access.

```
__global__ void multiplyMatrixTiled(int* A, int* B, int* C)
{
    __shared__ int sA[TileSize * TileSize];
    __shared__ int sB[TileSize * TileSize];

    // decode 2D block position from 1D block index
    int tilesPerRow = (N + TileSize - 1) / TileSize;  // = 4 for N=100
    int blockRow = blockIdx.x / tilesPerRow;
    int blockCol = blockIdx.x % tilesPerRow;

    // thread's position within the 32x32 tile
    int threadRow = threadIdx.x / TileSize;  // 0..31
    int threadCol = threadIdx.x % TileSize;  // 0..31

    // global row/col this thread is responsible for
    int row = blockRow * TileSize + threadRow;
    int col = blockCol * TileSize + threadCol;

    int sum = 0;

    // iterate over k-tiles
    for (int t = 0; t < (N + TileSize - 1) / TileSize; t++)
    {
        // each thread loads its one element into shared mem (row major)
        int aCol = t * TileSize + threadCol;
        sA[threadRow * TileSize + threadCol] = (row < N && aCol < N) ? A[row * N + aCol] : 0;

        int bRow = t * TileSize + threadRow;
        sB[threadRow * TileSize + threadCol] = (bRow < N && col < N) ? B[bRow * N + col] : 0;

        __syncthreads();

        // accumulate partial dot product from this k-tile
        for (int k = 0; k < TileSize; k++)
            sum += sA[threadRow * TileSize + k] * sB[k * TileSize + threadCol];

        __syncthreads();
    }

    if (row < N && col < N)
        C[row * N + col] = sum;
}
    if (row < N && col < N) C[row * N + col] = sum;
}
```

Listing 4: Matrix multiplication kernel – global + shared memory

2.1.4 Kernel 3 – Global + Constant Memory

Constant memory is a 64 KB read-only region that can broadcast a single value to all threads at once – but only when every thread reads the *same* address simultaneously. Because only two 64×64 tiles fit in the budget, the CPU must pack tiles, call `cudaMemcpyToSymbol`, and launch a kernel $(N/64)^3$ times, accumulating partial results with `atomicAdd`. Note that here the same logic was used as for shared memory, which was outside of the scope of the task but seemed interesting to implement instead.

```
__constant__ int constA[ConstTile * ConstTile];
__constant__ int constB[ConstTile * ConstTile];

__global__ void multiplyMatrixConstant(int* C,
    int tileRowOffset, int tileColOffset, int kOffset, int N)
{
    int abs_tid = blockIdx.x * blockDim.x + threadIdx.x;
    if (abs_tid >= ConstTile * ConstTile) return;
```

```
    int threadRow = abs_tid / ConstTile;  int threadCol = abs_tid % ConstTile;
    int globalRow = tileRowOffset + threadRow;
    int globalCol = tileColOffset + threadCol;
    if (globalRow >= N || globalCol >= N) return;
    int sum = 0;
    for (int k = 0; k < min(ConstTile, N - kOffset); k++)
        sum += constA[threadRow * ConstTile + k] * constB[k * ConstTile + threadCol];
    atomicAdd(&C[globalRow * N + globalCol], sum);
}

//in main:
    int tilesPerRow = (N + ConstTile - 1) / ConstTile;
    int tilesPerK   = (N + ConstTile - 1) / ConstTile;
    std::vector<int> tileA(ConstTile * ConstTile, 0);
    std::vector<int> tileB(ConstTile * ConstTile, 0);

    for (int tileRow = 0; tileRow < tilesPerRow; tileRow++) {
      for (int tileCol = 0; tileCol < tilesPerRow; tileCol++) {
        for (int tk = 0; tk < tilesPerK; tk++) {
          int rowOffset = tileRow * ConstTile;
          int colOffset = tileCol * ConstTile;
          int kOff      = tk    * ConstTile;

          // Tile A laden
          std::fill(tileA.begin(), tileA.end(), 0);
          for (int r = 0; r < ConstTile; r++)
            for (int c = 0; c < ConstTile; c++) {
              int gr = rowOffset + r, gc = kOff + c;
              if (gr < N && gc < N) tileA[r * ConstTile + c] = matrixA[gr * N + gc];
            }

          // Tile B laden
          std::fill(tileB.begin(), tileB.end(), 0);
          for (int r = 0; r < ConstTile; r++)
            for (int c = 0; c < ConstTile; c++) {
              int gr = kOff + r, gc = colOffset + c;
              if (gr < N && gc < N) tileB[r * ConstTile + c] = matrixB[gr * N + gc];
            }

          cudaMemcpyToSymbol(constA, tileA.data(), ConstTile * ConstTile * sizeof(int));
          cudaMemcpyToSymbol(constB, tileB.data(), ConstTile * ConstTile * sizeof(int));

          int constThreadsPerBlock = 256;
          int constBlocks = (ConstTile * ConstTile + constThreadsPerBlock - 1) /
              constThreadsPerBlock;
          multiplyMatrixConstant<<<constBlocks, constThreadsPerBlock>>>(
              d_result_const, rowOffset, colOffset, kOff);
          cudaDeviceSynchronize();
        }
      }
    }
```

Listing 5: Matrix multiplication kernel – global + constant memory

2.2 Results

All GPU results were verified against a CPU reference. Each size was measured over 10 timed runs preceded by a warm-up; the median is reported throughout.

2.2.1 Memory Migration Times

The one-time CPU→GPU transfer (Table 2) is flat at $\approx 0.13\,\text{ms}$ for small N due to fixed setup overhead, growing to $\approx 1.28\,\text{ms}$ only at $N = 1024$. The constant kernel's `cudaMemcpyToSymbol` calls are individually similar in cost, but are repeated $(N/64)^3$ times – 512 times at $N = 512$ and 4096 times at $N = 1024$ – so the accumulated transfer cost dominates its runtime entirely.

Table 2: CPU→GPU transfer time for two $N \times N$ matrices (median over 10 runs, RTX 2070).

N	CPU → GPU (ms)
128	≈ 0.13
256	≈ 0.32
512	≈ 0.92
1024	≈ 1.28

2.2.2 Kernel Execution Times

Table 3 lists median runtimes across all kernels. The figures below visualise the same data from different angles.

Table 3: Median kernel execution times (ms). Constant-memory times include all CPU staging and symbol transfers.

N	CPU	Global only	Global+Shared	Global+Constant
32	≈ 0.20	≈ 0.020	≈ 0.018	≈ 0.16
64	≈ 1.6	≈ 0.029	≈ 0.025	≈ 0.24
128	≈ 12.5	≈ 0.047	≈ 0.039	≈ 1.8
256	≈ 98	≈ 0.73	≈ 0.145	≈ 17
512	≈ 786	≈ 1.42	≈ 1.07	≈ 137
1024	≈ 6292	≈ 6.97	≈ 4.09	≈ 826

Figure 3 shows all kernels together on a log scale. The CPU is slowest throughout; the constant kernel's steep rise reflects $(N/64)^3$ staging cycles, not GPU computation; the gap is consistently larger from $N = 256$ onwards. Note that the H to D transfer time is excluded from the kernel timings.

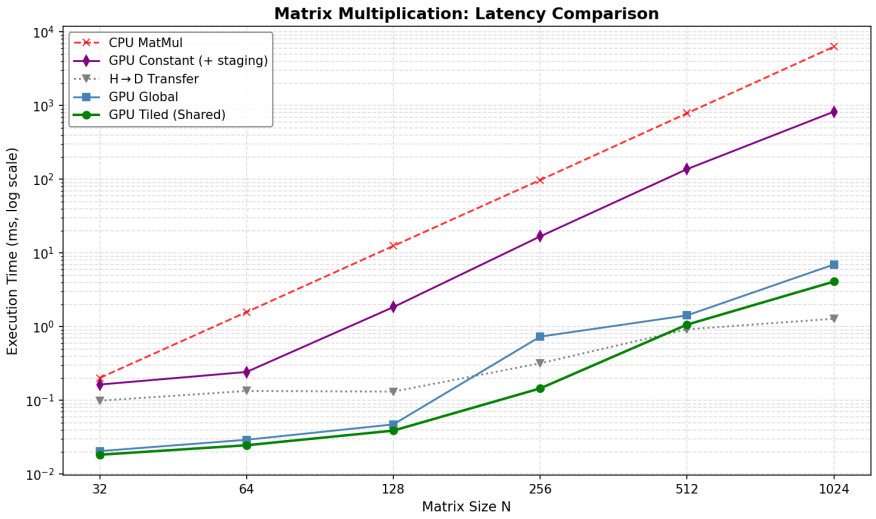


Figure 3: Execution time vs. N (log scale) for all kernels. All lines follow $O(N^3)$ (beyond $N = 128$). The constant-memory curve rises faster due to the cubically increasing number of staging cycles.

Figure 4 isolates the two main GPU kernels; the tiled kernel is consistently faster, with the gap widening beyond $N = 256$.

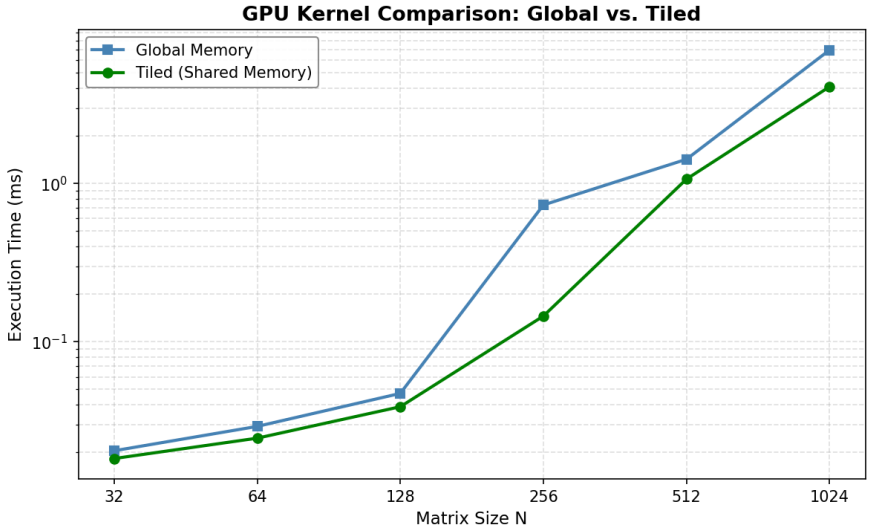


Figure 4: GPU global vs. tiled shared memory. The advantage of tiling grows as matrix size increases.

Figure 5 shows that the tiled kernel's speedup over the CPU grows from $\approx 11\times$ at $N = 32$ to $\approx 1540\times$ at $N = 1024$.

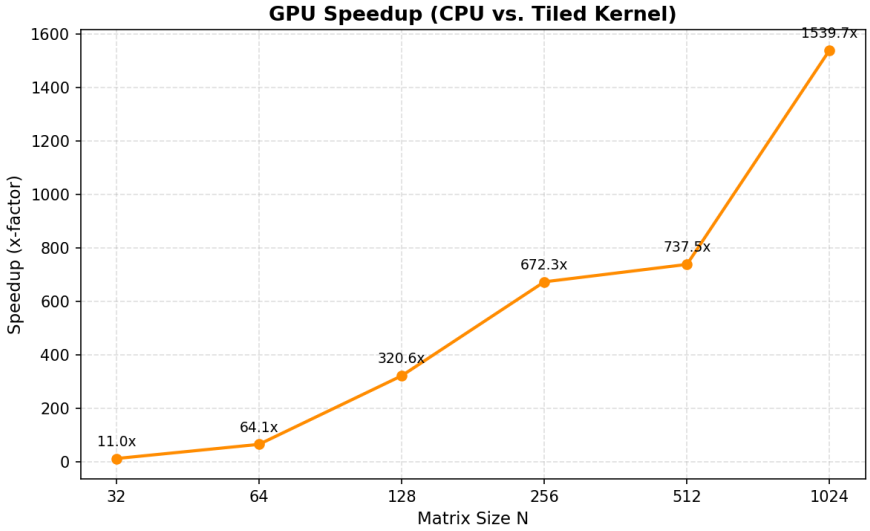


Figure 5: Speedup of the tiled kernel over the CPU. The advantage grows once there is enough work to keep the GPU fully occupied.

To understand *why* tiling helps more at larger N, a staging-only kernel was measured: it performs the same tile loads as the tiled kernel but skips the arithmetic, giving an upper bound on the time spent on data movement. Figure 6 shows this fraction drops from $\approx 70\%$ at $N = 32$ to $\approx 19\%$ at $N = 512$, where it plateaus. At small N the kernel is almost entirely memory-bound and shared memory brings little benefit; at large N computation dominates and the $32\times$ reduction in global reads pays off.

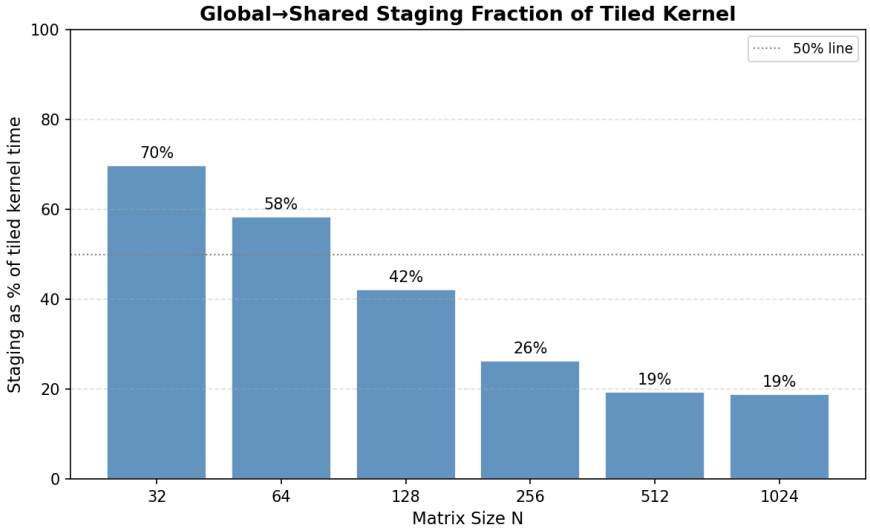


Figure 6: Fraction of the tiled kernel's total time spent loading tiles from global into shared memory. The shift from memory-bound to compute-bound explains why the tiled kernel's advantage grows with N.

2.3 Analysis

2.3.1 How Much Does Memory Migration Cost?

The one-time H→D transfer is not a bottleneck for the global or tiled kernels: at $N = 1024$ it costs $\approx$ 1.28 ms against kernel runtimes of 6.97 and 4.09 ms respectively. For the constant kernel the repeated staging completely dominates – 4096 pack-copy-launch cycles at $N = 1024$ account for almost all of its 826 ms total.

2.3.2 How Do the Three Memory Types Compare?

The tiled kernel is the fastest GPU option at every size. Its advantage over the global kernel is small at $N \leq 128$ (hardware caches mask the difference) but grows to 1.7× at $N = 1024$, as Figure 4 shows. The staging fraction (Figure 6) explains the trend: the 32× reduction in global reads only pays off once the kernel is compute-bound, which happens above $N \approx 512$.

The constant kernel is the slowest GPU option because the broadcast feature of constant memory never triggers – each thread reads a *different* address, so reads are no faster than global memory. The 64 KB size cap then forces the $(N/64)^3$ staging overhead that dominates the runtime.

2.3.3 How Do the Times Scale with Input Size?

All implementations follow $O(N^3)$, visible as a slope-3 line in Figure 3. At small N all GPU kernels sit near the same ≈ 0.02 ms floor due to fixed kernel-launch overhead.

2.4 Conclusions

The tiled shared-memory kernel is the best option for all practically relevant sizes, achieving $\approx 1540\times$ speedup over the CPU at $N = 1024$. Its advantage over the global kernel is modest at small N but grows to 1.7× at large N once the kernel becomes compute-bound and the reduced global memory traffic matters. The constant-memory kernel is a poor fit: the broadcast never fires in matrix multiplication, and the repeated tile staging makes it orders of magnitude slower than the other two GPU kernels. Constant memory is better suited to small, fixed, read-only data that all threads truly share – such as filter coefficients in a convolution.

References

[1] Anthropic. (2024). *Claude*. Large language model. Throughout this project, the model was utilised as an assistant for C++/CUDA code generation, debugging, clarification of technical concepts, and the structuring and refinement of the written report.